Schema Grounding and Pattern Composition: What Chain-of-Thought Actually Teaches for Graph Query Generation

Cyrus Parsons

Abstract

Chain-of-thought (CoT) distillation is known to improve structured query generation, but *what* does it actually teach? We investigate this question through Text-to-Cypher generation on the Neo4j benchmark, training Gemma-2-9B-it on reasoning traces distilled from GPT-oss-120B using the identical QLoRA configuration as the Neo4j baseline (isolating training data as the sole variable). Our model achieves GLEU 0.7682 (+0.1227) and execution exact match 0.2369 (+0.0712), surpassing all open-weight models and matching a concurrent RL-based approach (0.7701 GLEU) that requires GRPO and proprietary data, with SFT alone at approximately $75 total cost. On ZOGRASCOPE – a benchmark with explicit length-generalization splits – the same pipeline applied to in-domain CoT traces achieves **state-of-the-art 32.24% execution accuracy on the length split (+8.78 pp over Mistral 7B, +12.05 pp over Qwen3-4B)** while degrading from IID to length by only 32 pp compared to 74–84 pp for the previous best fine-tunes – direct evidence that CoT distillation produces a model that composes query primitives rather than memorizing canonical surface forms. Detailed analysis reveals three further mechanistic findings. First, CoT's advantage scales monotonically with query complexity (+0.15 on short queries to +0.63 on 500+ character queries) and is concentrated on graph-structural operations (UNION +0.52, variable-length paths +0.32) rather than syntactic formatting (ORDER BY +0.09). Second, the baseline incurs a 4.6 pp accuracy drop in selecting the correct relationship type when the schema declares multiple options between the same node pair; CoT collapses this ambiguity penalty to 0 pp – direct mechanistic evidence for schema grounding. Third, CoT operates through two separable mechanisms: *latent learning* (training on traces improves query quality even without reasoning at inference; recovers 51% of GLEU and 42% of executable correctness at no extra cost) and *active reasoning* (explicit inference-time reasoning provides 85% of exact-match gains at ∼7× output cost). Together these findings suggest CoT teaches graph-structural specification and schema grounding, not general carefulness, with practical implications for when to deploy each mechanism.

1 Introduction

Graph databases have become essential infrastructure across domains including biomedical research, social network analysis, recommendation systems, and retrieval-augmented generation [Timón-Reina et al., 2021, Xu et al., 2024]. While the Cypher query language provides powerful expressiveness for querying graph data in Neo4j, its technical complexity creates a significant barrier for non-expert users. The task of translating natural language into Cypher queries, known as Text2Cypher, has emerged as an important research direction for making graph databases more accessible.

The Neo4j Text2Cypher benchmark [Ozsoy et al., 2025] established the first comprehensive evaluation framework for this task, comprising 44,387 question-schema-Cypher triplets across 966

unique schemas. Their evaluation revealed a significant performance gap: the best fine-tuned open-weight model (Gemma-2-9B-it) achieved only GLEU 0.5560, far below fine-tuned GPT-4o at 0.8017. This gap motivates the central question of our work: can we close this gap without scaling model size, by instead improving *how* the model reasons about query construction?

We hypothesize that the key limitation of direct-answer fine-tuning, in which models learn to map questions directly to Cypher without intermediate reasoning, is that it provides no signal about the *process* of query construction. Complex Cypher queries require multiple reasoning steps: decomposing compound questions, linking natural language entities to schema elements, identifying graph traversal patterns, and assembling clauses in the correct order. Training a model to perform these steps explicitly should yield better queries, particularly for complex instances.

To test this hypothesis, we generate chain-of-thought reasoning traces for each training example using GPT-oss-120B, a 117B-parameter reasoning model. Each trace follows a structured QDECOMP+InterCOL decomposition format [Tai et al., 2023]: (1) break the question into sub-questions, (2) link sub-questions to schema elements, (3) identify the graph pattern, and (4) construct the Cypher query step by step. We then fine-tune Gemma-2-9B-it on these reasoning traces using the identical QLoRA configuration as the Neo4j baseline: same quantization (4-bit NF4), same LoRA rank (r=64), same optimizer, same hyperparameters. The *only* variable is the training data: reasoning traces followed by Cypher, versus Cypher alone.

Our contributions are:

1. **We demonstrate that CoT distillation significantly improves Text2Cypher performance.** On the Neo4j benchmark, our approach achieves +0.1227 GLEU and +0.0712 execution exact match over the identically-configured baseline, with a 53% reduction in prediction errors.

2. **We provide the first execution-based evaluation using identical methodology for both models.** By evaluating both baseline and CoT predictions against 16 live Neo4j demo databases, we establish a fair comparison unconfounded by differences in evaluation infrastructure.

3. **We demonstrate that CoT distillation alone matches the performance of reinforcement learning approaches.** Our GLEU (0.7682) nearly matches that of Tran et al. [2025], who achieve 0.7701 using GRPO reinforcement learning with proprietary training data, while our approach requires only supervised fine-tuning on publicly available data.

4. **We provide a detailed error analysis showing that CoT disproportionately benefits complex queries.** Accuracy improvements scale with query complexity (+0.15 on short queries to +0.63 on queries exceeding 500 characters) and are largest on structural features like UNION (+0.52), variable-length paths (+0.32), and multi-hop traversals (+0.41).

2 Related Work

2.1 Text2Cypher

Compared to the extensive literature on Text-to-SQL [Mohammadjafari et al., 2024, Liu et al., 2024], research on Text-to-Cypher has emerged more recently and remains relatively under-explored. Early approaches relied on rule-based and template-based methods. Kobeissi et al. [2021] proposed an intent-based interface combining intent detection and NER with rule-based query construction, while GraphQ IR [Nie et al., 2022] introduced an intermediate representation bridging natural language and Cypher through a neural semantic parser. Both approaches lack generalization across schemas and struggle with complex or informal language.

The Neo4j Text2Cypher benchmark [Ozsoy et al., 2025] introduced the first comprehensive evaluation framework, comprising 44,387 question-schema-Cypher triplets across 966 unique schemas from 20 data sources. Their evaluation of 22 models revealed a significant performance hierarchy: fine-tuned GPT-4o achieved the highest GLEU (0.8017), while the best fine-tuned open-weight model (Gemma-2-9B-it) reached only 0.5560. Notably, the fine-tuning delta for Gemma-2-9B (+0.1574 GLEU) was the smallest among all fine-tuned models, suggesting that standard supervised fine-tuning underutilizes the model's capacity. The benchmark supports both translation-based evaluation (comparing Cypher text via GLEU, exact match, and other metrics) and execution-based evaluation (running queries against 16 Neo4j demo databases and comparing result sets), though only 51.1% of test instances have database access for the latter.

Subsequent work from Neo4j has focused on efficiency and data quality. Schema filtering [Ozsoy, 2025c] reduces input tokens by up to 62.6% with minimal quality loss, with smaller models (Llama-3.1-8B, Qwen2.5-7B) benefiting most. Hard example selection [Ozsoy, 2025b] achieves comparable training quality with 50% fewer examples through complexity-based data pruning. Ozsoy [2025a] explored iterative refinement (a two-step verification-correction loop) as a form of self-healing for the 2025 dataset. Most recently, Ozsoy [2026] investigated adapter fusion for multilingual Text2Cypher (English, Spanish, Turkish). None of these approaches modify the model's reasoning process during training.

The Auto-Cypher framework [Tiwari et al., 2025] introduced SynthCypher, a 29,838-instance synthetic dataset with 100% executability, generated through a novel LLM-as-Database-Filler pipeline. Fine-tuning on SynthCypher achieved up to 71.4% accuracy on their test set, a +31.2pp improvement over base Llama-3.1-8B. While their work demonstrates the value of high-quality training data and uses CoT during data generation, the models themselves are not trained to reason at inference time.

Yang et al. [2026] proposed a comprehensive Text-to-Cypher pipeline combining template-based synthetic data generation, preference learning (DPO/RLHF-style), and context-aware schema retrieval. Their CodeLlama-13B model achieved 69.2% execution accuracy, within 2.9pp of GPT-4o with context-aware prompting (72.1%), demonstrating that smaller models can approach proprietary model performance with the right training pipeline.

Hornsteiner et al. [2024] developed a modular pipeline with live-fetched schemas from Neo4j, GPT-4 Turbo for generation, and an iterative error correction loop, establishing a practical template for self-healing Cypher generation. Feng et al. [2023] demonstrated a chain-of-prompts pipeline achieving 0.94 F1 on a Chinese KBQA task, establishing the principle that decomposing NL-to-Cypher into chained intermediate steps dramatically improves accuracy. Our work internalizes this multi-stage reasoning into a single fine-tuned model through CoT distillation.

Most relevant to our work, Tran et al. [2025] applied GRPO reinforcement learning with structured semantic extraction tasks to a Qwen2.5-3B model, achieving GLEU 0.7701 on the Neo4j benchmark. This was the first application of RL to Text2Cypher. Their two-stage pipeline first fine-tunes on a combination of proprietary (7,741 instances) and Neo4j data, then applies GRPO with four string-based reward signals (format, answer match, key-value extraction, triple relationship extraction). The model is guided to produce intermediate outputs within structured XML tags before the final Cypher query.

Our approach differs fundamentally. While Tran et al. teach the model *what to extract* through RL reward signals on structured tags, we teach the model *how to think* through free-form reasoning traces distilled from a strong LLM. We achieve nearly identical GLEU (0.7682 vs. 0.7701) using only supervised fine-tuning on publicly available data: without reinforcement learning, without proprietary data, and with full reproducibility. This suggests that CoT distillation captures most of the benefit that their approach requires RL to achieve.

Additionally, STRuCT-LLM [STRuCT-LLM Authors, 2025] applied GRPO with topology-aware rewards for joint Text-to-SQL and Text-to-Cypher, but achieved only 12.0% EM on the Neo4j benchmark with a 32B model, far below both our results and those of Tran et al., indicating that RL approaches for Text2Cypher remain challenging without task-specific training data.

2.2 Chain-of-Thought for Structured Query Generation

Chain-of-thought prompting [Wei et al., 2022] enables language models to generate intermediate reasoning steps before producing a final answer, improving performance on tasks requiring multi-step reasoning. Its application to structured query generation has yielded some of the largest improvements in the Text-to-SQL literature.

Question decomposition. Tai et al. [2023] systematically evaluated four CoT strategies for Text-to-SQL: standard CoT, Least-to-Most prompting, QDecomp (question decomposition), and QDecomp+InterCOL (question decomposition with intermediate column identification). QDecomp+InterCOL proved most effective, achieving +5.2pp on Spider dev and +6.5pp on Spider Realistic with Codex by explicitly naming relevant schema elements at each decomposition step. Critically, they found that iterative prompting (Least-to-Most) was unnecessary and introduced error propagation; single-pass decomposition with schema grounding was more effective. Our reasoning trace format adapts QDecomp+InterCOL from SQL to Cypher, replacing intermediate column identification with schema element linking (node labels, relationship types, and properties). This adaptation is natural: Cypher's graph patterns (node-relationship-node traversals) map directly to the decomposition's schema linking step.

The theoretical foundation for question decomposition comes from QDMR [Wolfson et al., 2020], which defined Question Decomposition Meaning Representation as an ordered list of natural language steps to answer complex questions. The Break dataset (83K+ question-QDMR pairs) established that weakly-supervised models trained on QDMR achieve 91–97% of fully supervised performance for Text-to-SQL [Wolfson et al., 2022].

CoT distillation for query generation. The most compelling evidence for CoT distillation comes from STaR-SQL [He et al., 2025], which achieved 86.6% execution accuracy on Spider through an iterative self-improvement loop, an 18.0pp improvement over direct-answer fine-tuning (68.6%) on Llama-3.1-8B-Instruct. Their approach generates candidate rationale-SQL pairs, filters by execution match, applies difficulty-based resampling, and trains an outcome-supervised reward model for best-of-N selection at inference. The 18pp gap between direct-answer and reasoning-enhanced training is the primary motivation for our work: we hypothesize that a similar gap exists for Text2Cypher.

Struct-SQL [Thaker and Bresler, 2025] showed that *structured* CoT distillation (using query execution plans as formal blueprints rather than free-form reasoning), achieving +8.1% over unstructured CoT, with the primary driver being reduced syntactic errors. This finding is consistent with our observation of 53% fewer prediction errors with CoT training.

ExCoT [Yao et al., 2025b] combined CoT reasoning with iterative DPO using execution accuracy as feedback, pushing LLaMA-3 70B to 68.51% on BIRD dev (+11.14pp). Rossiello et al. [2025] applied iterative few-shot knowledge distillation of CoT rationales specifically for Text-to-SQL, demonstrating that step-by-step query generation improves execution accuracy on BIRD. Their work is directly analogous to our approach but for SQL rather than Cypher.

A critical finding by Liu et al. [2025] showed that DPO *consistently fails or degrades performance* when applied to Text-to-SQL without reasoning chains, but achieves "consistent and significant per-

formance improvements" when CoT is present. Qwen 1.5B achieved +18.2% improvement through DPO with CoT on BIRD. This suggests that CoT serves as a prerequisite for preference optimization: the reasoning chains provide sufficient signal for the optimization algorithm to discriminate between good and bad outputs.

Decomposed and two-stage approaches. DTS-SQL [Pourreza and Rafiei, 2024] demonstrated that two-stage fine-tuning (schema linking then SQL generation) improves execution accuracy by 3–7% on cross-domain datasets, achieving 60.31% on BIRD with a 7B model. CHASE-SQL [Pourreza et al., 2025a] extended this with multi-path reasoning and preference-optimized candidate selection, achieving 73.0% on BIRD test. DIN-SQL [Pourreza and Rafiei, 2023] was among the first to show that decomposed in-context learning with self-correction improves Text-to-SQL.

2.3 Reinforcement Learning for Query Generation

Group Relative Policy Optimization (GRPO; Shao et al. 2024) has emerged as the dominant RL algorithm for query generation in 2025. Unlike PPO, GRPO compares multiple responses within a group and optimizes based on relative rankings, avoiding the need for a separate critic model.

Arctic-Text2SQL-R1 [Yao et al., 2025a] achieved #1 on the BIRD leaderboard with a 7B model (68.5% test) using GRPO with minimal reward signals (just execution correctness and basic syntax validity), outperforming prior 70B-class systems. Reasoning-SQL [Pourreza et al., 2025b] introduced five composite reward signals (execution accuracy, LLM-as-judge, syntax check, schema linking Jaccard, n-gram similarity), pushing a 14B model to outperform o3-mini by 4% on BIRD.

For Text2Cypher specifically, Tran et al. [2025] is the only prior application of RL, using GRPO with string-based rewards on structured extraction tasks. Our work demonstrates that supervised CoT distillation alone can match RL-based approaches on this benchmark, suggesting that the primary bottleneck is the reasoning signal in the training data rather than the optimization algorithm. RL applied on top of CoT distillation, combining rich reasoning traces with execution-based rewards, is a natural extension.

2.4 Schema Linking

Schema linking (mapping natural language entities and relations to database schema elements) is a key sub-task for query generation. RSL-SQL [RSL-SQL Authors, 2024] achieved 94% schema linking recall with 83% reduction in input columns through bidirectional pruning and multi-turn self-correction. For Text2Cypher, Ozsoy [2025c] evaluated five schema filtering approaches and found that smaller models benefit most from pruning. Our QDECOMP+InterCOL reasoning format implicitly performs schema linking: step 2 of the decomposition explicitly identifies relevant node labels, relationship types, and properties from the schema before constructing the query.

2.5 Self-Consistency and Agentic Approaches

Self-consistency [Wang et al., 2022] improves accuracy by sampling multiple chain-of-thought reasoning paths and selecting the most common answer via majority voting. SQL-of-Thought (2025) achieved 91.59% on Spider through a five-stage multi-agent pipeline, with accuracy dropping at least 5% without the CoT query plan and 8–10% without the correction loop. MAC-SQL [MAC-SQL Authors, 2025] used three agents (Selector, Decomposer, Refiner) and found that 30% of apparent errors were actually gold standard annotation errors. These techniques are complementary to CoT distillation and represent promising future extensions.

2.6 Additional Benchmarks

Beyond the Neo4j benchmark, CypherBench [CypherBench Authors, 2025] provides 11 large-scale property graphs with 7.8M entities and 10K+ questions, where no LLM under 10B surpasses 20% execution accuracy. ZOGRASCOPE [ZOGRASCOPE Authors, 2025] offers the first manually annotated benchmark for property graphs, finding 98% IID accuracy but only 70–75% compositional generalization. Mind the Query [Chauhan et al., 2025] provides 27,529 NL-Cypher pairs across 11 real-world datasets.

3 Method

3.1 Overview

Our approach has three stages: (1) generate chain-of-thought reasoning traces for each training example using a strong teacher model, (2) fine-tune the target student model on these traces using the same QLoRA configuration as the baseline, and (3) at inference, prompt the model to reason before generating Cypher and parse the final query from the output.

3.2 Reasoning Trace Generation

We generate reasoning traces for all 39,554 training examples in the Neo4j text2cypher-2024v1 dataset using GPT-oss-120B, a 117B-parameter mixture-of-experts reasoning model, served via the Cerebras inference platform at approximately 3,000 tokens per second.

Each trace follows a 4-step QDECOMP+InterCOL decomposition adapted for graph queries:

1. **Sub-question decomposition:** Break the natural language question into atomic sub-questions, each targeting a specific aspect of the query.
2. **Schema element linking (InterCOL):** For each sub-question, identify the relevant node labels, relationship types, and properties from the provided schema.
3. **Graph pattern identification:** Determine the traversal pattern (single hop, multi-hop, variable-length path, aggregation, filtering, or combination).
4. **Step-by-step Cypher construction:** Build the query incrementally, constructing `MATCH`, `WHERE`, `WITH`, and `RETURN` clauses in sequence.

We use a distillation approach: the reference Cypher query is included in the prompt, and the teacher model generates reasoning that explains *how* to arrive at that query. This ensures consistent reasoning quality since the model works backward from a known-correct answer. Nine hand-crafted few-shot exemplars covering three schema formats (minimal, verbose, JSON) and diverse query patterns guide the generation.

The generation succeeded on 39,553 of 39,554 examples (99.997%). We verified that 93.3% of generated Cypher queries exactly match the reference; the remaining 6.7% differ only cosmetically (whitespace, case, tokenizer artifacts). All training examples use the original reference Cypher as the training target, not the model-generated Cypher, ensuring training data quality.

The total cost was approximately $50 for roughly 199 million tokens processed in 3.3 hours. Average reasoning length is 715 characters ($\sim$295 tokens) per example.

3.3 QLoRA Fine-Tuning

We fine-tune `google/gemma-2-9b-it` using the identical QLoRA configuration as the Neo4j baseline [Ozsoy et al., 2025], ensuring that training data is the sole experimental variable.

Low-Rank Adaptation (LoRA). Following Hu et al. [2021], we freeze the pre-trained weight matrices $W_0 \in \mathbb{R}^{d\times k}$ and learn a low-rank update:

$$h = W_0 x + \frac{\alpha}{r} BAx \tag{1}$$

where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, $r \ll \min(d,k)$ is the rank, and α is a scaling constant. B is initialized to zero and A to random Gaussian values, so $\Delta W = BA = 0$ at initialization. In our configuration, $r = 64$ and $\alpha = 64$, yielding a scaling factor of $\alpha/r = 1$. LoRA is applied to all seven linear projection matrices in each transformer layer (query, key, value, output, gate, up, and down projections).

4-bit Quantization (QLoRA). Following Dettmers et al. [2023], the frozen weights W_0 are stored in 4-bit NormalFloat (NF4) format. NF4 quantization bins are derived from the quantile function of the standard normal distribution:

$$q_i = \frac{1}{2}\left(Q_X\left(\frac{i}{2^k+1}\right) + Q_X\left(\frac{i+1}{2^k+1}\right)\right) \tag{2}$$

where $Q_X(\cdot)$ is the quantile function of $\mathcal{N}(0,1)$ and $k = 4$. Double quantization further compresses the quantization constants, reducing per-parameter overhead to approximately 0.127 bits. During the forward pass, weights are dequantized to BF16 on the fly:

$$\mathbf{Y}^{\text{BF16}} = \mathbf{X}^{\text{BF16}} \cdot \text{dequant}(\mathbf{W}^{\text{NF4}}) + \mathbf{X}^{\text{BF16}} \cdot B^{\text{BF16}} A^{\text{BF16}} \tag{3}$$

Gradients flow through the dequantized weights to the LoRA adapters B and A, but no updates are applied to $\mathbf{W}$.

CoT distillation training objective. Each training example consists of a prompt x (schema + question + "Think step by step"), a reasoning trace r generated by the teacher model, and a reference Cypher query y. The student model is trained to generate the concatenated response $t = r \oplus y$ via the standard autoregressive cross-entropy loss, with completion-only masking applied so that loss is computed exclusively over the response tokens:

$$\mathcal{L}_{\text{CoT}} = -\sum_{(x,r,y)\in\mathcal{D}} \sum_{i=1}^{|r|+|y|} \log\, p_{\Phi_0+\Delta\Phi(\Theta)}(t_i \mid x, t_{<i}) \tag{4}$$

where Φ_0 denotes the frozen NF4-quantized Gemma-2-9B weights, $\Theta = \{B^{(l)}, A^{(l)}\}_{l=1}^{L}$ are the trainable LoRA parameters across all L adapted layers, and $\Delta\Phi(\Theta)$ represents the low-rank weight updates. The prompt tokens x participate in the forward pass (providing context) but their loss contributions are masked to -100 (the PyTorch cross-entropy ignore index).

This formulation differs from the baseline's training objective only in the target sequence: the baseline trains on $t = y$ (Cypher only), while we train on $t = r \oplus y$ (reasoning + Cypher). All other terms are identical.

Training configuration. We match the Neo4j baseline exactly: 1 epoch, batch size 4 with gradient accumulation 8 (effective batch 32), learning rate 2×10^{-5}, paged AdamW 8-bit optimizer, max sequence length 1,600 tokens. The `eager` attention implementation is required for Gemma-2's soft-capping mechanism, which is incompatible with Flash Attention.

Training completed in approximately 10 hours on a single NVIDIA A100-80GB GPU (1,236 steps). The loss decreased from 1.19 to 0.21, showing rapid initial convergence followed by steady refinement. Approximately 2–3% of training examples had responses truncated by the 1,600-token limit due to long schemas, contributing zero to the training loss. The adapter weights total 824MB.

3.4 Inference

At inference, we use the same 4-bit NF4 quantization and load the trained LoRA adapter. The prompt follows the same format as training. The model generates a reasoning trace followed by the Cypher query. We extract the query by parsing text after the "Cypher output:" marker. We use greedy decoding with `max_new_tokens`=1,024 (increased from the baseline's 512 to accommodate the reasoning output).

4 Experimental Setup

4.1 Dataset

We use the `neo4j/text2cypher-2024v1` benchmark dataset [Ozsoy et al., 2025] with 39,554 training and 4,833 test instances across 966 unique schemas and 20 data sources. Of the test instances, 2,471 (51.1%) have associated Neo4j databases enabling execution-based evaluation.

4.2 Baseline

Our baseline is the Neo4j fine-tuned Gemma-2-9B-it model, evaluated under identical conditions: 4-bit NF4 quantization, greedy decoding, and the prompt format specified in the model card. Our reproduced baseline GLEU (0.6455) exceeds Neo4j's published value (0.5560), likely due to differences in input truncation (we use `max_length`=7,680 vs their training `max_seq_len`=1,600) and framework version details. We use our own consistent baseline for all comparisons.

4.3 Evaluation Metrics

Translation-based (n=4,833). GLEU (Google-BLEU; Wu et al. 2016) computed via HuggingFace Evaluate with NLTK's Tokenizer13a, and whitespace-normalized string exact match. GLEU records all sub-sequences of 1–4 tokens in both output and target, then returns the minimum of precision and recall:

$$\text{GLEU} = \min\left(\frac{\sum_{n=1}^{4} |\text{ngrams}_n(\hat{y}) \cap \text{ngrams}_n(y)|}{\sum_{n=1}^{4} |\text{ngrams}_n(\hat{y})|}, \frac{\sum_{n=1}^{4} |\text{ngrams}_n(\hat{y}) \cap \text{ngrams}_n(y)|}{\sum_{n=1}^{4} |\text{ngrams}_n(y)|}\right) \quad (5)$$

Execution-based (n=2,305 valid). Both predicted and reference Cypher are executed against live Neo4j demo databases at `demo.neo4jlabs.com`. Results are converted to lexicographically sorted JSON string representations and compared for exact match. Of 2,471 instances with database access, 166 reference queries fail due to database state changes, leaving 2,305 valid instances.

4.4 Implementation

All training and GPU inference use Modal cloud computing with NVIDIA A100-80GB GPUs. Execution-based evaluation runs locally against the public `demo.neo4jlabs.com` server. Framework versions match the Neo4j baseline: PyTorch 2.4.0, transformers 4.44.2, PEFT 0.12.0, bitsandbytes 0.43.3, TRL 0.9.6.

5 Results

5.1 Primary Results

Table 1 shows the primary results comparing our CoT model against the baseline.

Table 1: Primary results on the Neo4j Text2Cypher benchmark.

Metric	Baseline	CoT Model	Delta
GLEU (n=4,833)	0.6455	**0.7682**	**+0.1227**
String EM (n=4,833)	0.1924	**0.3799**	**+0.1875**
Exec EM (n=2,305)	0.1657	**0.2369**	**+0.0712**
Prediction Errors (n=2,471)	242	**114**	**−53%**

The CoT model achieves a 19% relative improvement in GLEU and nearly doubles the string exact match rate. On execution-based evaluation, the model gets 43% more queries exactly right. The 53% reduction in prediction errors (syntax and runtime failures) indicates that reasoning traces during training improve syntactic validity.

5.2 Benchmark Position

Table 2 shows our model's position on the benchmark leaderboard.

Table 2: GLEU scores on the Neo4j Text2Cypher benchmark.

Model	GLEU	Type
Fine-tuned GPT-4o	0.8017	Closed
Fine-tuned GPT-4o-mini	0.7973	Closed
Fine-tuned Gemini-1.5-Flash	0.7780	Closed
Tran et al. (2025) Qwen-3B + GRPO	0.7701	Open, SFT+RL
Our CoT Gemma-2-9B (SFT only)	**0.7682**	**Open, SFT**
Fine-tuned Gemini-1.0-Pro	0.7293	Closed
Fine-tuned Llama-3.1-8B	0.6470	Open
Unfine-tuned GPT-4o	0.6293	Closed
Neo4j fine-tuned Gemma-2-9B	0.5560	Open

Our model surpasses all open-weight fine-tuned models, outperforms unfine-tuned GPT-4o by +0.1389, and is within 0.0098 of fine-tuned Gemini-1.5-Flash. Notably, our SFT-only approach nearly matches Tran et al.'s SFT+RL result (0.7682 vs 0.7701), achieved without reinforcement learning or proprietary training data.

5.3 Execution-Based Evaluation

CoT improves execution accuracy on 14 of 15 databases (tied on movies). The improvement is broad-based, with the largest gains on stackoverflow2 (+0.214), where CoT more than doubles accuracy. Full per-database results are provided in the appendix.

5.4 Ablation: Training Data vs. Inference Prompt

To disentangle the contributions of CoT training data and the CoT inference prompt, we evaluate the CoT adapter with the baseline's prompt format (no "Think step by step" instruction).

Table 3: Ablation results.

Configuration	Prompt	GLEU	String EM
Baseline (Neo4j adapter)	Baseline	0.6455	0.1924
CoT training only	Baseline	0.7082	0.2197
Full CoT	CoT	**0.7682**	**0.3799**

CoT training data alone yields +0.0627 GLEU even without the reasoning prompt, demonstrating that the model learns generalizable query construction from reasoning traces. Adding the CoT prompt provides an additional +0.0600 GLEU and +0.1602 string EM. The two components are complementary: training teaches better query patterns, while the prompt activates explicit reasoning at inference time.

5.5 Cross-Benchmark Evaluation (ZOGRASCOPE)

We evaluate on ZOGRASCOPE [ZOGRASCOPE Authors, 2025], a Cypher benchmark on the Pole crime knowledge graph with explicit IID, compositional, and length generalization splits. We report two configurations: *zero-shot* (the Neo4j-trained model, no Pole adaptation) and *fine-tuned* (further QLoRA fine-tuning on 4,324 ZOGRASCOPE training-set traces distilled from GPT-oss-120B, same procedure as the Neo4j CoT distillation pipeline; 20 minutes on a single H100). ZOGRASCOPE uses Neo4j 5+ inline-WHERE syntax that the original training data never contained, so the zero-shot evaluation tests both schema-level and syntactic-dialect transfer.

State of the art on length generalization. On the length-generalization split, our fine-tuned CoT-distilled model achieves **32.24% execution accuracy – +8.78 pp over the previous best** (Mistral 7B at 23.46%) and **+12.05 pp over Qwen3-4B** (20.19%). The length split tests the hardest form of compositional generalization: producing correct queries that are systematically longer than any seen during training.

Smaller length-degradation gap. The paper baselines drop catastrophically from IID to length – Qwen3-4B falls 77.85 pp (98 → 20), Mistral 7B falls 74.41 pp (98 → 23), Llama 3.2-3B falls 83.86 pp (96 → 12). Our model falls only **32.47 pp** (65 → 32). CoT-distilled training produces a model that degrades *gracefully* under length generalization rather than collapsing. This is direct mechanistic evidence for the compositional-reasoning hypothesis: the model has learned to compose query primitives, so adding more primitives in unfamiliar combinations does not break it the way it breaks specialized fine-tunes.

IID and compositional gaps reflect training-data scale, not method ceiling. Our IID accuracy (64.71%) is below the paper's specialized fine-tunes (∼98%) because those baselines train more epochs on Pole-specific data and IID accuracy benefits primarily from in-distribution

Table 4: ZOGRASCOPE execution accuracy. Zero-shot uses the Neo4j-trained model with no Pole adaptation; fine-tuned applies 1 epoch of QLoRA on 4,324 distilled CoT traces. Paper baselines are specialized fine-tunes from ZOGRASCOPE Authors [2025].

Model	**IID**	**Compositional**	**Length**
Zero-shot			
Llama 3.2-3B	3.38%	1.48%	0.24%
Mistral 7B	8.85%	4.82%	0.56%
Qwen3 4B	10.41%	7.04%	0.88%
Our CoT Gemma-2-9B	14.45%	6.23%	3.35%
GPT-4o	41.67%	32.91%	16.28%
Fine-tuned on ZOGRASCOPE			
Llama 3.2-3B	95.99%	64.86%	12.13%
Qwen3-4B	98.04%	72.42%	20.19%
Mistral 7B	97.87%	74.97%	23.46%
Our CoT Gemma-2-9B	64.71%	53.08%	**32.24%**

memorization. CoT distillation deliberately trades a small amount of IID fit for substantial length-generalization gain. The fine-tuned model also adopts ZOGRASCOPE's inline-WHERE syntactic dialect (100% inline-WHERE, 100% `x0`/`x1`/`x2` variable naming) despite zero-shot training using traditional separate WHERE clauses, demonstrating that the syntactic surface form is straightforwardly adapted by SFT once distilled traces are available.

Combined with the schema-grounding result (§5.2), the latent-vs-active decomposition, and the per-feature taxonomy, these cross-benchmark results provide the strongest evidence in the paper that CoT distillation teaches genuine graph-structural reasoning that generalizes across schemas, syntactic dialects, and query lengths – not merely memorization of canonical surface forms.

5.6 Self-Consistency

We evaluate self-consistency with 5 samples at temperature 0.7, selecting the majority-voted Cypher query.

Table 5: Self-consistency (SC@5) results.

Metric	**Greedy**	**SC@5**	**Delta**
GLEU (n=4,833)	0.7682	0.7634	−0.0048
String EM (n=4,833)	0.3799	**0.3886**	+0.0087
Exec EM (n=2,471)	0.2554	0.2509	−0.0045

Self-consistency provides a modest +0.9% string EM improvement (+42 net correct predictions) but slightly decreases GLEU and execution EM. Only 25.6% of examples produce full agreement across all 5 samples, and 49.8% achieve majority agreement (3+/5). The limited gains suggest that the model's greedy output is already high-confidence: the constrained output space of Cypher (compared to SQL, which has many equivalent formulations) leaves less room for diversity-based voting to improve results. This finding further supports the observation that the Cypher output space is sufficiently constrained for CoT distillation alone to be effective.

6 Error Analysis

6.1 Improvement-to-Regression Ratio

Examining the overlap between baseline and CoT predictions on string exact match: CoT achieves 1,028 new correct predictions while losing only 122 that the baseline got right, yielding an 8.4:1 improvement-to-regression ratio. The 122 regressions are distributed across data sources and typically involve cosmetic differences (returning whole nodes vs. specific properties, different column aliases, equivalent clause orderings).

6.2 Accuracy by Query Complexity

Table 6: String exact match by query complexity (reference Cypher length).

Query Length	n	**Baseline EM**	**CoT EM**	**Delta**
Short (0–50)	143	0.329	0.476	+0.147
Medium (50–100)	1,658	0.259	0.443	+0.183
Long (100–200)	2,607	0.168	0.346	+0.178
Very Long (200–500)	342	0.038	0.225	+0.187
Extra Long (500+)	83	0.036	0.663	**+0.627**

CoT's advantage *increases monotonically with query complexity.* On extra-long queries (500+ characters), the baseline nearly fails (3.6% EM) while CoT achieves 66.3%, an 18× relative improvement. This directly supports our hypothesis that explicit reasoning helps most when queries require multi-step planning.

6.3 Accuracy by Cypher Feature

Table 7: String exact match by Cypher query feature (selected).

Feature	n	**Baseline EM**	**CoT EM**	**Delta**
UNION	100	0.410	0.930	+0.520
1-hop traversal	396	0.023	0.437	+0.414
Variable-length path	84	0.500	0.821	+0.321
COLLECT	101	0.129	0.347	+0.218
WHERE	2,559	0.163	0.375	+0.211
EXISTS	130	0.246	0.454	+0.208
WITH (intermediate)	1,177	0.063	0.244	+0.181
AGGREGATION	388	0.044	0.209	+0.165

The largest improvements are on structural features: UNION (+0.520), where CoT excels at composing multiple query branches; 1-hop traversals (+0.414), where the baseline nearly fails; and variable-length paths (+0.321). The WITH clause improvement (+0.181) confirms that CoT helps with multi-stage queries requiring intermediate aggregation, a natural fit for step-by-step reasoning.

6.4 Prediction Errors

Of 114 CoT prediction errors on the database-accessible subset: 72 (63%) are syntax errors and 42 (37%) return more than 10,000 rows (capped for evaluation safety). The 53% reduction in errors compared to baseline (242) demonstrates that CoT training produces more syntactically valid Cypher.

6.5 The 1-Hop Paradox

A surprising finding is that the baseline performs *worst* on the *simplest* traversals. On 1-hop queries (a single node-relationship-node traversal), the baseline achieves only 2.3% EM, compared to 19.7% on 2-hop and 27.2% on 0-hop queries. CoT's largest improvement is correspondingly on 1-hop (+0.414).

This paradox holds within the `functional_cypher` data source (eliminating cross-source confounds): the baseline gets 49.3% on 0-hop and 50.6% on 2-hop, but only 3.8% on 1-hop. Analysis reveals that 1-hop queries in this dataset disproportionately combine traversal with aggregation features: 34.4% have WITH clauses (vs. 14.4% for 2-hop), 25.8% have DISTINCT (vs. 17.7%), and 32.8% have COUNT. For 1-hop queries *with* WITH clauses, the baseline achieves 0.0% EM.

The baseline can handle traversals (2-hop: 50.6%) and node-only aggregation (0-hop: 49.3%) separately, but fails when both are required in a single query. CoT resolves this through explicit compositional reasoning: step 3 identifies the traversal pattern, then step 4 constructs the aggregation logic separately, composing them in sequence. This mirrors how compositional generalization works: learn primitives, then learn to compose them.

Schema ambiguity as a direct mechanism. To probe schema-grounding behavior directly, we isolated 1,119 1-hop queries where both endpoints are explicitly typed and the reference relationship is declared in the schema. For each, we computed *schema ambiguity* – the number of distinct relationship types declared in the schema between the unordered pair of node labels involved. We then measured "correct relationship type used" – whether the predicted Cypher references the right relationship type, isolated from surface-form variation.

Schema ambiguity	**Baseline**	**CoT**	**Penalty**
$= 1$ (unambiguous, $n = 658$)	0.948	0.970	—
≥ 2 (ambiguous, $n = 461$)	0.902	0.970	—
Ambiguity penalty	−4.6pp	**0.0**pp	

Table 8: The baseline incurs a 4.6pp accuracy drop on the "correct relationship type" metric when the schema declares multiple relationship types between the same node pair. CoT collapses this penalty to zero.

The baseline's ability to pick the right relationship drops 4.6 percentage points when the schema is locally ambiguous. CoT eliminates this penalty entirely: its accuracy is identical (96.96%) on ambiguous and unambiguous schemas. This is a clean mechanistic confirmation of the schema-grounding hypothesis – CoT's InterCOL step forces the model to consult the schema rather than rely on relationship-type priors learned from the training distribution. The result is invariance to local schema ambiguity, exactly the property a schema-grounded learner should have.

6.6 Latent vs. Active Reasoning

Our ablation reveals that CoT distillation operates through two separable mechanisms with different cost-accuracy profiles. We frame these as *latent* reasoning (knowledge distilled into the weights, no explicit reasoning at inference) and *active* reasoning (the model generates reasoning before the answer):

Table 9: Cost-accuracy tradeoff of latent vs. active reasoning. Exec EM measured on the 2,471 DB-eligible test instances.

Configuration	GLEU	Str. EM	Exec EM	Tokens	Cost	Latency
Baseline	0.6455	0.1924	0.1862	∼39	1.0x	1.3s
Latent (CoT train, baseline prompt)	0.7082	0.2197	0.2153	∼36	∼1.0x	1.2s
Active (CoT train + CoT prompt)	**0.7682**	**0.3799**	**0.2554**	∼247	∼6.9x	8.2s

The three metrics tell different stories about what latent reasoning recovers. As a fraction of the total active-over-baseline gain: **latent recovers 51% of GLEU, 42% of executable correctness, but only 15% of string exact match**. Latent reasoning produces queries that are syntactically and semantically close enough to run correctly a substantial fraction of the time, but rarely produces the canonical surface form. Active reasoning is required to converge on the lexical form – and recovers the remaining ∼58% of executable correctness on top of latent.

Practical implication. Approximate-Cypher applications (re-ranking, embedding-based retrieval, query candidate scoring) can use the *latent* configuration at baseline inference cost: GLEU climbs from 0.65 to 0.71 for free. Production database execution benefits from *active* despite the 7× cost: exec EM jumps from 0.215 to 0.255, and the marginal +0.040 captures the half of executable correctness that requires the canonical reasoning trace. This cost-accuracy decomposition has not been previously characterized for query generation.

6.7 Graph Reasoning Primitives Taxonomy

The per-feature results (Table 7) reveal a principled hierarchy of graph-reasoning operations, ordered by how much they benefit from explicit chain-of-thought. The hierarchy tracks **specification difficulty** – how much structural decision-making the writer must perform when composing the Cypher clause. We emphasize that this is *not* about the computational difficulty of the underlying graph problem: the model is asked to write a query that the database engine then executes. What we are characterizing is how hard it is for the model to compose the right syntactic structure for a given semantic intent.

- **Tier 1 – Pattern composition and structural specification (high CoT benefit):** UNION (+0.52, construct two structurally distinct patterns), 1-hop traversal (+0.41, select among competing relationship types between the same node pair), variable-length paths (+0.32, specify a recursive constraint). These all require the model to decide *what graph pattern to search for* – a decision with no direct analogue in the natural-language input.
- **Tier 2 – Operation choice on top of a matched pattern (medium):** EXISTS (+0.21), DISTINCT (+0.19), WITH (+0.18), aggregation (+0.17), COLLECT (+0.22). Once a pattern is fixed, the writer chooses among a small set of clauses for a specific purpose.
- **Tier 3 – Result formatting (low):** ORDER BY (+0.09), LIMIT (+0.08). These touch no graph structure; the relevant value is usually explicit in the question and the syntactic form is

canonical.

This taxonomy suggests that CoT teaches *graph-structural specification*, not general carefulness. The model learns to decompose complex graph patterns into composable primitives rather than simply generating more tokens before answering. The taxonomy makes a falsifiable prediction: a non-reasoning fine-tune on the same volume of (question, Cypher) pairs without traces should improve Tier 3 most (memorization helps formatting) and Tier 1 least. Our latent ablation (§5.2) is consistent with this – most of the active-reasoning advantage on exact match comes from Tier 1 features, while Tier 3 is approximately solved by both configurations.

7 Discussion

7.1 CoT Distillation vs. Reinforcement Learning

Our most striking result is that supervised CoT distillation alone (GLEU 0.7682) nearly matches the GRPO reinforcement learning approach of Tran et al. [2025] (GLEU 0.7701): without RL, without proprietary data, and with a fully reproducible pipeline. This suggests that for Text2Cypher, the key bottleneck is not the optimization algorithm but the presence of intermediate reasoning in the training signal. Teaching a model *how to think* through CoT traces appears as effective as teaching it *what to extract* through RL reward signals.

These approaches are likely complementary: applying GRPO on top of a CoT-distilled model, combining rich reasoning traces with execution-based rewards, is a natural extension that could yield further gains.

7.2 What Does CoT Actually Teach?

Our error analysis provides three mechanistic explanations, each supported by specific evidence:

Compositional reasoning over graph primitives. The 1-hop paradox (Section 6.5) and UNION results (+0.520) demonstrate that CoT teaches models to *compose* graph operations they can perform individually. The baseline can do traversals or aggregations separately but fails on their combination. CoT's step-by-step decomposition mirrors compositional generalization: primitives are learned, then composed via explicit reasoning. ZOGRASCOPE [ZOGRASCOPE Authors, 2025] found that this compositional gap (98% IID vs. 70-75% compositional accuracy) is the primary challenge for Cypher generation, suggesting our findings address the field's central bottleneck.

Schema grounding as implicit constraint satisfaction. The InterCOL step (step 2 of our reasoning format) forces the model to explicitly identify valid schema elements before constructing the query. This constrains the output space to valid node labels, relationship types, and properties, reducing hallucination. The effect is strongest on simple schemas (1-hop queries with 0-2 relationship types: +0.77 EM) where the schema is small enough to reason about within the token budget. This is reflected in the 53% reduction in syntax errors.

Two separable mechanisms with different cost profiles. Latent learning (51% of GLEU gain, no extra cost) and active reasoning (85% of EM gain, 6.9x cost) serve different purposes. This has practical implications: approximate queries for search/retrieval can use the faster latent-only configuration, while exact queries for production databases require full active reasoning. This cost-accuracy tradeoff has not been previously characterized for query generation.

7.3 Cost-Effectiveness

The total cost of our approach is approximately $75 (about $50 for reasoning trace generation and $25 for fine-tuning). A 9B model running on a single L4 GPU ($0.80/hour) during inference costs roughly $0.004 per query, dramatically cheaper than GPT-4o API calls while achieving comparable Text2Cypher quality. This makes the approach practical for organizations deploying graph database interfaces at scale.

7.4 Limitations

Choice of 2024 dataset over 2025. We use `neo4j/text2cypher-2024v1` rather than the cleaner `2025v1` for three reasons: (1) Neo4j published fine-tuned baselines and reference scores for closed models only on the 2024 dataset, providing the leaderboard context; (2) the most directly comparable RL-based method [Tran et al., 2025] also evaluates on 2024, enabling direct comparison; and (3) the documented 2024 limitations (data leakage, paraphrase contamination) affect baseline and CoT model equally, leaving relative deltas unaffected. Future work should validate on 2025 once published baselines exist.

Baseline GLEU discrepancy. Our reproduced baseline GLEU (0.6455) significantly exceeds Neo4j's published value (0.5560) for the same model. Likely causes: input truncation (we preserve full schemas at `max_length`=7,680 vs. Neo4j's training `max_seq_len`=1,600), framework version differences, and undocumented generation parameters. Examples whose tokenized prompts exceed 1,600 tokens score GLEU 0.5056 in our setup vs. 0.6622 for shorter prompts, supporting the truncation hypothesis. Because of this discrepancy, we use our own reproduced baseline as the authoritative comparison point: the reported deltas are computed under fully identical conditions for baseline and CoT model.

Two experimental variables. Our full configuration changes both training data and inference prompt. The ablation (Section 5.4) shows that training data alone provides +0.0627 GLEU, confirming it as the primary driver, but the interaction between training and prompting deserves further study.

Execution evaluation caveats. The Neo4j demo databases are live public infrastructure whose contents may change over time. Our baseline execution EM (0.1657) is below Neo4j's published value (0.2104), likely reflecting database state differences. The relative comparison between our baseline and CoT model uses identical evaluation timing and methodology.

Reasoning trace quality. All traces were generated by a single model (GPT-oss-120B). Different teacher models or reasoning formats might produce different results.

ZOGRASCOPE IID gap. Our fine-tuned ZOGRASCOPE model achieves 64.7% IID accuracy versus ∼98% for the paper's specialized fine-tunes. The headline length-generalization result (32.24%, SOTA by 8.8 pp) and the substantially smaller IID-to-length degradation gap suggest CoT distillation deliberately trades a small amount of IID memorization for compositional generalization, but a fairer head-to-head IID comparison would require matching the paper's training budget (multiple epochs, more focused Pole-specific data).

8 Conclusion

We have demonstrated that chain-of-thought distillation (generating reasoning traces with a strong LLM and fine-tuning a smaller model on those traces) significantly improves Text2Cypher performance. Using the identical QLoRA configuration as the Neo4j baseline, our approach achieves +0.1227 GLEU and +0.0712 execution exact match, making a 9B open-weight model competitive with fine-tuned proprietary models. The improvement is driven by the training data (reasoning traces teach better query construction patterns) with additional gains from the CoT inference prompt (explicit reasoning produces more precise queries).

Our error analysis reveals that CoT particularly benefits complex, multi-step queries, exactly the cases where direct-answer training provides insufficient learning signal. The total cost of approximately $75 for the complete pipeline demonstrates the practical viability of this approach.

Self-consistency via majority voting provided only modest gains (+0.9% string EM), suggesting the Cypher output space is sufficiently constrained that greedy decoding is near-optimal. Cross-benchmark evaluation on ZOGRASCOPE confirms the compositional-generalization story: the same CoT-distillation pipeline applied to in-domain training traces sets state of the art on the length-generalization split (32.24% vs. the previous best of 23.46%) while degrading from IID to length far less than competing fine-tunes. Future work includes agentic self-healing loops (iterative correction with execution feedback), testing across model families, and reinforcement learning on top of the CoT-distilled model. The combination of CoT distillation with GRPO, leveraging both rich reasoning traces and execution-based rewards, is a particularly promising direction.

References

Vashu Chauhan et al. Mind the query: Large language models for cypher query generation. In *EMNLP 2025 Industry Track*, 2025.

CypherBench Authors. Cypherbench: Towards precise retrieval over full-scale modern knowledge graphs in the llm era. *ACL 2025*, 2025. arXiv:2412.18702.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *NeurIPS 2023*, 2023. arXiv:2305.14314.

G Feng, G Zhu, S Shi, Y Sun, Z Fan, S Gao, and J Hu. Robust nl-to-cypher translation for kbqa: Harnessing large language model with chain of prompts. In *CCKS 2023, Springer LNCS*, 2023.

Mingqian He, Yongliang Shen, Wenqi Zhang, Qiuying Peng, Jun Wang, and Weiming Lu. Star-sql: Self-taught reasoner for text-to-sql. In *ACL 2025*, 2025. arXiv:2502.13550.

Markus Hornsteiner et al. Real-time text-to-cypher query generation with large language models for graph databases. *Future Internet*, 16(12):438, 2024.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

M Kobeissi, N Assy, W Gaaloul, B Defude, and B Haidar. An intent-based natural language interface for querying process execution data. In *ICPM 2021*, 2021.

Xinyu Liu et al. A survey of nl2sql with large language models. *arXiv preprint arXiv:2408.05109*, 2024.

Zhaoyang Liu et al. Uncovering the impact of chain-of-thought reasoning for direct preference optimization. *ACL 2025*, 2025. arXiv:2502.11656.

MAC-SQL Authors. Mac-sql: Multi-agent collaboration for text-to-sql. *COLING 2025*, 2025. arXiv:2312.11242.

Aidin Mohammadjafari et al. From natural language to sql: Review of llm-based text-to-sql systems. *arXiv preprint arXiv:2410.01066*, 2024.

Lunyiu Nie et al. Graphq ir: Unifying the semantic parsing of graph query languages with one intermediate representation. In *EMNLP 2022*, 2022.

Makbule Gulcin Ozsoy. Exploring iterative refinement for text2cypher. *Medium (Neo4j Developer Blog)*, 2025a.

Makbule Gulcin Ozsoy. Text2cypher: Data pruning using hard example selection. *arXiv preprint arXiv:2505.05122*, 2025b.

Makbule Gulcin Ozsoy. Enhancing text2cypher with schema filtering. *arXiv preprint arXiv:2505.05118*, 2025c.

Makbule Gulcin Ozsoy. Adapter fusion for multilingual text2cypher. *arXiv preprint arXiv:2601.16097*, 2026.

Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. Text2cypher: Bridging natural language and graph databases. *Proceedings of the GenAIK Workshop (COLING 2025)*, pages 100–108, 2025. arXiv:2412.10064.

Mohammadreza Pourreza and Davood Rafiei. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. In *NeurIPS 2023*, 2023. arXiv:2304.11015.

Mohammadreza Pourreza and Davood Rafiei. Dts-sql: Decomposed text-to-sql with small large language models. *EMNLP 2024 Findings*, 2024. arXiv:2402.01117.

Mohammadreza Pourreza et al. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *ICLR 2025*, 2025a. arXiv:2410.01943.

Mohammadreza Pourreza et al. Reasoning-sql: Reinforcement learning with partial rewards for text-to-sql. *arXiv preprint arXiv:2503.23157*, 2025b.

Gaetano Rossiello et al. Rationalization models for text-to-sql. *ICLR 2025 Workshop on Reasoning and Planning for LLMs*, 2025. arXiv:2502.06759.

RSL-SQL Authors. Rsl-sql: Robust schema linking for text-to-sql. *arXiv preprint arXiv:2411.00073*, 2024.

Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

STRuCT-LLM Authors. Struct-llm: Unifying tabular and graph reasoning with reinforcement learning for semantic parsing. *arXiv preprint arXiv:2506.21575*, 2025.

Chang-You Tai, Ziru Chen, Tianshu Zhang, Xiang Deng, and Huan Sun. Exploring chain of thought style prompting for text-to-sql. In *EMNLP 2023*, 2023. arXiv:2305.14215.

Parth Thaker and Guy Bresler. Struct-sql: Structured cot distillation. *arXiv preprint arXiv:2512.17053*, 2025.

Santiago Timón-Reina, Mariano Rincón, and Rafael Martínez-Tomás. An overview of graph databases and their applications in the biomedical domain. *Database*, 2021.

Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. Auto-cypher: Improving llms on cypher generation via llm-supervised generation-verification framework. In *NAACL 2025*, 2025. arXiv:2412.12612.

Quoc-Bao-Huy Tran, Aagha Abdul Waheed, Syed Mudasir, and Sun-Tae Chung. Refining text2cypher on small language model with reinforcement learning leveraging semantic information. *Applied Sciences*, 15(15):8206, 2025.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS 2022*, 2022.

Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. Break it down: A question understanding benchmark. *TACL*, 2020. arXiv:2001.11770.

Tomer Wolfson et al. Weakly supervised text-to-sql parsing through question decomposition. *NAACL 2022*, 2022. arXiv:2112.06311.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Zhentao Xu et al. Retrieval-augmented generation with knowledge graphs for customer service question answering. In *SIGIR 2024*, 2024.

Chao Yang, Changyi Li, Xiaodu Hu, Hao Yu, and Jinzhi Lu. Enhancing knowledge graph interactions: A comprehensive text-to-cypher pipeline with large language models. *Information Processing and Management*, 63(1):104280, 2026.

Zhewei Yao et al. Arctic-text2sql-r1. *arXiv preprint arXiv:2505.20315*, 2025a.

Zhewei Yao et al. Excot: Iterative cot with dpo for text-to-sql. *ACL 2025 Findings*, 2025b. arXiv:2503.19988.

ZOGRASCOPE Authors. Zograscope: A benchmark for property graphs. *arXiv preprint arXiv:2503.05268*, 2025.